PREDICTING THE VIRALITY OF YOUTUBE VIDEOS

YIXUAN BAO

JINGHUI CHEN

URIEL SANTA CRUZ

June 12th

SPRING 2025

1. Introduction

The rapid growth of video platforms like YouTube has transformed how content gains traction online. In the age of algorithm-driven platforms, content creators and marketers alike are constantly seeking ways to understand and predict the performance of their YouTube videos. This project's final goal is to provide a data-driven tool that offers predictive insights into a video's viral potential, based solely on information available at or near the time of upload.

2. Problem Statement

YouTube videos vary widely in performance, with some achieving viral success and others struggling to attract views. We think predicting whether a video will become popular is a challenging task due to the wide range of influencing factors, such as content quality, timing, channel reputation, and viewer engagement. This project formulates the problem as a three-class classification task: Low popularity; Medium popularity; High popularity. Given a set of metadata features available at upload time (e.g., title, publish time, tags, and video cover), the problem is to build a predictive model that can classify a video into one of these three categories.

3. Goals

Our initial approach involves creating a labeled dataset based on view thresholds and training a machine learning model to predict virality levels.

Project Objectives:

- Data Collection & Preprocessing
  Collect comprehensive trending video data from YouTube via the YouTube Data API v3, including metadata such as video titles, thumbnails, view counts, comment counts, publication timestamps, and category/tag information.
- Feature Extraction, Engineering & Exploratory Analysis
  Preprocess raw metadata into structured numerical features suitable for machine learning applications. Conduct exploratory data analysis (EDA), including features visualizations, to uncover potential relationships and validate the rationality of the predefined popularity classification scheme (low/medium/high).
- Model Development & Comparison
  Implement and evaluate multiple classification models, beginning with interpretable baseline approaches (e.g., Logistic Regression) and progressing to more complex models (e.g., Random Forest, Neural Networks). Also, investigate multimodal modeling strategies that integrate both textual (e.g., titles/tags) and visual (e.g., thumbnail) features to enhance predictive performance. Optimize model hyperparameters through systematic validation.
- Model Evaluation & Interpretability
  Assess model performance using standard metrics (accuracy, F1-score, confusion matrices) and statistical validation techniques. Identify and interpret the most influential features driving popularity predictions to derive actionable insights.

- Prediction System Deployment (Extension)
  If time permits, develop an end-to-end pipeline capable of ingesting a YouTube video URL, extracting and transforming its metadata into model-compatible features, and generating a predicted popularity class.

Project Structure Expectation:

```
YouTube_Hit_Predictor/
├── data/
│   └── youtube_trending.csv
├── notebooks/
│   ├── 01_data_exploration_visualization.ipynb
│   ├── 02_feature_engineering.ipynb
│   ├── 03_model_training.ipynb
│   └── 04_model_evaluation.ipynb
├── src/
│   ├── data_processing.py
│   ├── modeling.py
│   └── modules.py
└── README.md
```

4. Progress

4.1 Dataset

https://developers.google.com/youtube/v3

Utilization of a dataset containing 500 trending videos using the YouTube Data API v3. This quantity is sufficient for training and evaluating a classification model for the midterm report.

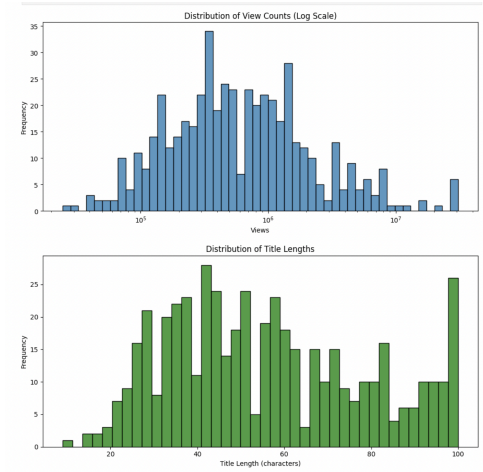
https://www.kaggle.com/datasets/datasnaek/youtube-new

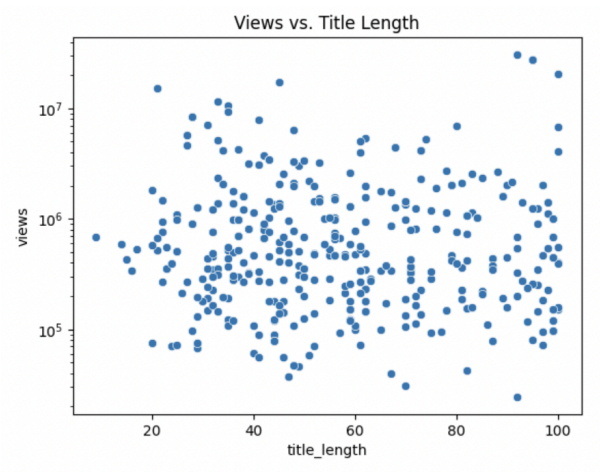
The Trending YouTube Video Statistics dataset on Kaggle, curated by user *datasnaek*, offers a comprehensive collection of daily trending video data from YouTube across multiple countries. This dataset is instrumental for analyzing content trends, user engagement, and regional preferences on the platform. For the purposes of visualization, the focus will be on the U.S..

4.2 Visualization & Feature Selection

- We try to use log-transformation for normalization. The log-scale distributions of views (Figure 1) reveal a heavy-tailed pattern, with most videos receiving moderate engagement ($<10^5$ interactions) while a small fraction achieves viral status ($>10^7$)
- Weak positive correlation between title length and views.

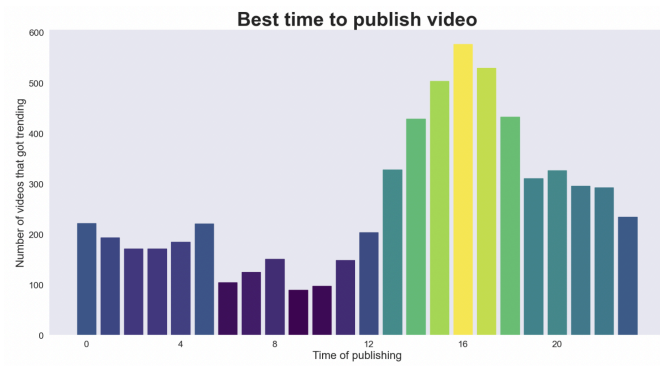
- Stronger correlation between hour_of_day and views than title length. Peak engagement time period at 14:00–18:00

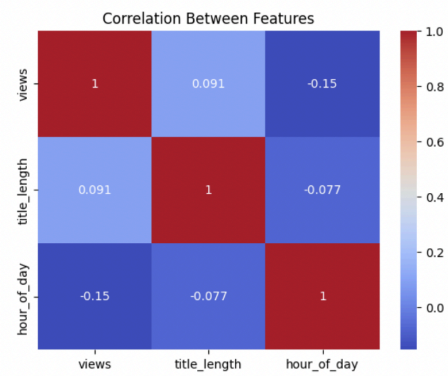





```
In [15]: corr = df[['views', 'title_length', 'hour_of_day']].corr()
         sns.heatmap(corr, annot=True, cmap='coolwarm')
         plt.title('Correlation Between Features')

Out[15]: Text(0.5, 1.0, 'Correlation Between Features')
```





- Features selection: The input features used in this project are derived from metadata fields that are typically available at or near the time of a video's upload.

  1) The title of the video, a text field, is processed to extract features such as character length (potential including presence of attention-grabbing keywords). These engineered features help model the potential impact of headline-style phrasing on viewership.

  2) The thumbnail, which is a visual preview image of the video, is optionally processed using computer vision techniques. From each thumbnail, we can extract features such as color distribution, brightness, text presence, and aesthetic quality scores (e.g., using pretrained models like Inception or CLIP). While this feature is more computationally expensive to handle, it can significantly enhance the model's ability to capture the "click appeal" of a video.

  3) The publish time, originally a timestamp, is decomposed into components such as hour of the day, day of the week, and a binary indicator of whether the video was posted on a weekend. This allows the model to capture potential time-related trends in video performance.

4) The category ID, a categorical variable representing the genre of the video (e.g., music, education, comedy), is encoded using one-hot encoding to make it suitable for input into the model.

- Visual: 512-d ResNet-18 embeddings from video thumbnails (L2-normalized).
- Engagement: views, likes, and comment counts → log(×+1) → Z-score.The Engagement set of features are based on previous data analysis and visualization as we see a clear skew in the distribution.

4.3 Current Model

- Logistic regression served as a fast and interpretable baseline, helping us select strong features.

```python
import torch.nn as nn

class MixedMLP(nn.Module):
    def __init__(self, input_dim=515):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, 256),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(256, 64),
            nn.ReLU(),
            nn.Linear(64, 1),
            nn.Sigmoid()  # use sigmoid for this one
        )

    def forward(self, x):
        return self.net(x)
import torch
from torch.utils.data import TensorDataset, DataLoader

# just the pipeline
X_tensor = torch.tensor(X, dtype=torch.float32)
y_tensor = torch.tensor(y.reshape(-1, 1), dtype=torch.float32)

dataset = TensorDataset(X_tensor, y_tensor)
loader = DataLoader(dataset, batch_size=16, shuffle=True)

# loss opt
model = MixedMLP(input_dim=X.shape[1]).to(device)
criterion = nn.BCELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
```

```python
#train
for epoch in range(10):
    model.train()
    total_loss = 0
    for xb, yb in loader:
        xb, yb = xb.to(device), yb.to(device)
        pred = model(xb)
        loss = criterion(pred, yb)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        total_loss += loss.item()
    print(f"Epoch {epoch+1}, Loss: {total_loss:.4f}")

model.eval()
with torch.no_grad():
    pred = model(X_tensor.to(device)).cpu().numpy()
    pred_class = (pred > 0.5).astype(int)

from sklearn.metrics import classification_report
print(classification_report(y, pred_class))
```

- The mixed model (which is still under refinement), on the other hand, was designed to capture more complex, non-linear relationships—especially from image features. While its theoretical capacity is higher, we also observed empirically better performance on our dataset, with improved accuracy compared to the baseline. Our ultimate goal is to further enhance the structure with more modalities such as text features including video titles and fine-tuning the model architecture for better generalization.

4.4 Current Result

```
Downloaded: fNtLIcyjsnI
Downloaded: _IzB3-CmtGA
Downloaded: YlxCVbrFS9M0
Downloaded: arc5h2TYS14
Downloaded: -2RVw2_QyxQ
Downloaded: gEHCXl4J9Qo
Downloaded: -OKSi1_YYfc
Downloaded: b4VTZx_Atfk
Downloaded: kOnQocd79BY
Downloaded: e4FAptEcS5c
Downloaded: ZAK4omx9uMU
Downloaded: xfmipNU4Odc
Downloaded: jp9HK-jY6yY
Downloaded: R6huWjJR-cM
/opt/anaconda3/lib/python3.12/site-packages/torchvision/models/_utils.py:208: UserWarning: The parameter 'pretrained' is deprecated
nce 0.13 and may be removed in the future, please use 'weights' instead.
  warnings.warn(
/opt/anaconda3/lib/python3.12/site-packages/torchvision/models/_utils.py:223: UserWarning: Arguments other than a weight enum or 'No
  for 'weights' are deprecated since 0.13 and may be removed in the future. The current behavior is equivalent to passing 'weights=R
Net18_Weights.IMAGENET1K_V1'. You can also use 'weights=ResNet18_Weights.DEFAULT' to get the most up-to-date weights.
  warnings.warn(msg)
100%|██████████| 100/100 [00:03<00:00, 66.12it/s]
              precision    recall  f1-score   support

           0       0.80      0.80      0.80        15
           1       0.40      0.40      0.40         5

    accuracy                           0.70        20
   macr0 avg       0.60      0.60      0.60        20
weighted avg       0.70      0.70      0.70        20
```

```
[4]: model.eval()
     with torch.no_grad():
         pred = model(X_tensor.to(device)).cpu().numpy()
         pred_class = (pred > 0.5).astype(int)

     from sklearn.metrics import classification_report
     print(classification_report(y, pred_class))

              precision    recall  f1-score   support

           0       0.95      0.96      0.95        77
           1       0.86      0.83      0.84        23

    accuracy                           0.93       100
   macro avg       0.91      0.89      0.90       100
weighted avg       0.93      0.93      0.93       100

[ ]:
```

4.5 Challenge & Solutions

- Data scraping

  Obtaining high-quality data has been one of our major challenges. While we have explored numerous well-established datasets, we opted to use an API that provides daily trending viral videos. Our long-term objective is to leverage this data as test cases to evaluate whether our model can successfully predict real-world examples. In the midterm phase, our dataset is not yet sufficiently representative—even when pushing the model to its limits. As a result, we will present this aspect of our progress in the final report.

```python
import pandas as pd
from googleapiclient.discovery import build

api_key = 'AIzaSyAJoE01-cg6OR053gcxW_Zf-FbOEvrWFvI'
youtube = build('youtube', 'v3', developerKey=api_key)

region_codes = ['US', 'IN', 'GB', 'BR', 'CA', 'DE', 'FR', 'AU', 'JP', 'MX']
all_data = []

for region in region_codes:
    print(f"Fetching region: {region}")
    request = youtube.videos().list(
        part='snippet,statistics',
        chart='mostPopular',
        regionCode=region,
        maxResults=50
    )
    response = request.execute()

    for item in response['items']:
        title = item['snippet']['title']
        views = int(item['statistics'].get('viewCount', 0))
        published = item['snippet']['publishedAt']
        category = item['snippet'].get('categoryId', 'unknown')
        all_data.append({
            'region': region,
            'title': title,
            'views': views,
            'publishedAt': published,
            'categoryId': category
        })

df = pd.DataFrame(all_data)
df.to_csv('youtube_trending_all_regions.csv', index=False)
print(f"✅ Saved {len(df)} videos from {len(region_codes)} regions.")
```

```python
df = pd.read_csv('youtube_trending_all_regions.csv')
```

	region	title	views	publishedAt	categoryId
0	US	YoungBoy Never Broke Again - Finest [Official...	1291316	2025-05-21T23:48:39Z	10
1	US	Marvel Rivals: Season 2.5 Kick Off! | Dev Visi...	465329	2025-05-22T15:01:17Z	20
2	US	Wendyyy - Picasso (Official Video)	1395411	2025-05-21T19:03:34Z	10
3	US	Apex predator found dead; Is the Everglades fi...	153640	2025-05-20T23:04:14Z	25
4	US	Stephen A. reacts to the Knicks' Game 1 loss 😱...	406689	2025-05-22T15:22:36Z	17
...	...	...	...	...	...
495	MX	El Peor Tour del Mundo	946314	2025-05-17T20:01:08Z	24
496	MX	🟢 ENCONTRÉ el TÚNEL del RATÓN de CHUCK E CHEES...	1433443	2025-05-17T19:10:00Z	24
497	MX	Kary Torres Oficial está en vivo Kary Pao Wend...	288059	2025-05-17T02:03:00Z	24
498	MX	QUEDO IMPRESIONADO CON ESTA GRAN CIUDAD DE CHI...	1015070	2025-05-16T01:03:40Z	22
499	MX	dando vueltas por japón	549417	2025-05-16T20:00:31Z	20

500 rows × 5 columns

5. Future Work

- Dataset Expansion & Refinement: The current analysis was conducted on a limited dataset. Future work should incorporate a larger collection of videos to enhance model robustness.
- Enhanced Visualization: Could be refined to better highlight key patterns and relationships.
- Classification Scheme Optimization: The current binary classification could be expanded into a three-tiered system (low/medium/high) to provide more accurate insights. The revised thresholds should be statistically justified, potentially helping the models performance.
- Further experimentation with feature transformations and selection. Dimensionality reduction techniques (e.g., PCA for correlated features), which should be explored to optimize the input feature space.
- The preliminary mixed-model approach (text + image features) warrants deeper investigation, including architectural refinements and expanded data modalities.

6. Updated Timeline & Team Roles

6.1 Timeline

Week 7: Train and evaluate baseline classification models using a subset of the dataset. Conduct initial performance assessments to identify strengths and limitations of the selected approaches.

Week 8: Refine classification criteria. Optimize model architectures and hyperparameters through systematic validation. Finalize the dataset

Week 9: Generate comprehensive visualizations to illustrate key findings, including feature importance, decision boundaries, and performance metrics. Interpret results in the context.

Week 10: Finalize code and complete the final report, incorporating analytical results, methodological details, and limitations. Prepare and rehearse the final presentation to effectively communicate project outcomes.

6.2 Member Roles

Uriel Santa Cruz – Integrated the YouTube Data API, automated metadata collection, and produced key data visualizations for insight and interpretability.

Bobby Bao – Designed and implemented mixed data model, conducted data analysis and model evaluation, and pipeline coordination.

Jinghui Chen – Led data acquisition via scraping and performed feature engineering tasks including cleaning, transformation, and feature selection.